SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS: **Total Marks: 20**

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.
(a) Identify key components required in a navigation bar for an e-commerce website.
(b) Design a responsive navigation bar using appropriate CSS techniques.
(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
(d) Evaluate the usability and suggest improvements for better user experience.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.
(a) Identify suitable CSS properties for hover effects.
(b) Suggest appropriate **CSS animation or transition** for product interaction.
(c) Design a hover effect for a product card.
(d) Evaluate performance and user experience considerations.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.
(a) Identify layout requirements of a dashboard (rows, columns, alignment).
(b) Compare **Flexbox and Grid** based on layout needs.
(c) Choose the most suitable layout technique and justify your choice.
(d) Evaluate the scalability and responsiveness of the chosen approach.

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.
(a) Identify key sections of a blog layout.
(b) Design a mobile-first layout using CSS.
(c) Demonstrate how the layout scales for larger screens using media queries.
(d) Evaluate accessibility and readability of the design.

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.
(a) Identify appropriate JavaScript events for menu interaction.
(b) Design event handling logic for menu toggle functionality.
(c) Implement interaction for user actions (click/hover).
(d) Evaluate performance and suggest improvements.

Code of Conduct:

I LOSHINI.L 192571015___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
LOSHINI.L

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts

		requirements and concepts			
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Question 1: Responsive Navigation Bar Design (4 Marks)

(a) Key components of an e-commerce navigation bar

- **Logo / brand name:** links to the home page and builds brand identity.
- **Primary category links:** Men, Women, Electronics, Home, Deals, etc.
- **Search bar:** the most-used element on a shopping site; supports product search.
- **Cart icon with item count badge:** quick access to checkout.
- **User account / login / wishlist:** personalisation and order tracking.
- **Hamburger toggle (mobile):** collapses the menu on small screens.
- **Optional:** language/currency selector, notifications, mega-menu for sub-categories.

(b) Responsive navigation bar – HTML and CSS

index.html

```
<header class="navbar">
<a href="#" class="logo">ShopEase</a>
<button class="menu-toggle" id="menuToggle"
aria-label="Toggle menu" aria-expanded="false">&#9776;</button>
<nav class="nav-links" id="navLinks">
<a href="#">Home</a>
<a href="#">Categories</a>
<a href="#">Deals</a>
<a href="#">Contact</a>
</nav>
<form class="search"><input type="search" placeholder="Search products..."></form>
<div class="actions"><a href="#">Account</a><a href="#">Cart (2)</a></div>
</header>
```

style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; } body
{ font-family: Arial, sans-serif; }

.navbar {
  display: flex; flex-wrap: wrap; align-items: center;
  gap: 12px; padding: 12px 20px;
  background: #1f3864; color: #fff;
  position: sticky; top: 0; z-index: 100;
}
.logo { font-size: 1.5rem; font-weight: bold; color: #fff; text-decoration: none; }
.nav-links { display: flex; gap: 18px; }
.nav-links a, .actions a { color: #fff; text-decoration: none; padding: 8px 4px; }
.nav-links a:hover { border-bottom: 2px solid #ffc107; }
.search { flex: 1; min-width: 160px; }
.search input { width: 100%; padding: 8px 12px; border-radius: 20px; border: none; }
.actions { display: flex; gap: 14px; }
.menu-toggle { display: none; background: none; border: none; color:
               #fff; font-size: 1.8rem; cursor: pointer; }

/* Tablet */
@media (max-width: 992px) {
  .navbar { gap: 8px; padding: 10px 14px; }
  .nav-links { gap: 10px; font-size: .9rem; }
}

/* Mobile */
@media (max-width: 768px) {
  .menu-toggle { display: block; margin-left: auto; order: 2; }
  .logo { order: 1; }
  .search { order: 3; flex-basis: 100%; }
  .actions { order: 4; flex-basis: 100%; justify-content: space-between; }
  .nav-links { order: 5; flex-basis: 100%; flex-direction: column;
               max-height: 0; overflow: hidden; transition: max-height .35s ease; }
  .nav-links.open { max-height: 300px; }
  .nav-links a { padding: 12px 0; border-top: 1px solid rgba(255,255,255,.2); }
```

```
}
```

(c) How the navigation adapts to smaller screens

Device	Behaviour
Desktop (> 992px)	Logo, links, search bar and cart appear in one horizontal row using Flexbox. Hamburger icon is hidden.
Tablet (768–992px)	Spacing and font size are reduced so every item still fits in a single row.
Mobile (< 768px)	Links are hidden; the hamburger icon appears. Search and cart move to full-width rows. Tapping the icon adds the class 'open', and the links slide down vertically.

script.js (toggle)

```
const toggle = document.getElementById('menuToggle');
const links  = document.getElementById('navLinks');
toggle.addEventListener('click', () => {
const open = links.classList.toggle('open');
  toggle.setAttribute('aria-expanded', open);
});
```

(d) Usability evaluation and improvements

Strengths: sticky header keeps navigation always reachable; search is prominent; the hamburger menu saves space; ARIA attributes support screen readers; touch targets are padded.

Suggested improvements:

- Add a mega-menu or dropdowns for sub-categories on desktop.
- Add search auto-suggest and recent searches.
- Close the menu automatically when a link is tapped or the user clicks outside.
- Provide a visible keyboard focus style (:focus-visible) and a 'skip to content' link.
- Ensure touch targets are at least 44×44 px and contrast ratio is at least 4.5:1.
- Use sticky bottom navigation on mobile for Home / Cart / Account (thumb-friendly).

Question 2: Product Hover Effect (4 Marks)

(a) Suitable CSS properties for hover effects

- **transform:** scale(), translateY(), rotate(): lifts or zooms the element.
- **transition:** animates property changes smoothly (duration, easing, delay).
- **box-shadow:** adds depth when the card is raised.
- **opacity / filter:** fade, brightness, blur or grayscale effects.
- **background-color, border-color:** colour feedback on buttons.
- **cursor:** pointer indicates interactivity.

(b) Suitable transition / animation

A CSS transition is best for hover because it is simple, reversible and runs on the compositor. The card lifts by translateY(-8px), the image zooms slightly with scale(1.08), and a shadow grows. An 'Add to Cart' button fades and slides up from the bottom. Duration 0.3s with ease-in-out feels natural. @keyframes animation can be used for a looping 'sale badge' pulse.

(c) Product card hover design

HTML

```
<div class="product-card">
<div class="img-wrap"><img src="shoe.jpg" alt="Running shoe"></div>
<h3>Running Shoes</h3>
<p class="price">&#8377;2,499</p>
<button class="add-btn">Add to Cart</button>
</div>
```

CSS

```
.product-card {
position: relative; width: 260px; padding: 16px; background:
  #fff; border-radius: 12px; overflow: hidden; box-shadow:
  0 2px 6px rgba(0,0,0,.12);
transition: transform .3s ease, box-shadow .3s ease;
}
.img-wrap { overflow: hidden; border-radius: 8px; }
.img-wrap img { width: 100%; display: block; transition: transform .4s ease; }

.add-btn {
width: 100%; padding: 10px; border: none; border-radius: 6px;
  background: #ff6f00; color: #fff; cursor: pointer;
opacity: 0; transform: translateY(15px); transition:
  opacity .3s ease, transform .3s ease;
}

.product-card:hover, .product-card:focus-within {
  transform: translateY(-8px);
box-shadow: 0 14px 28px rgba(0,0,0,.25);
}
.product-card:hover .img-wrap img { transform: scale(1.08); }
.product-card:hover .add-btn,
.product-card:focus-within .add-btn { opacity: 1; transform: translateY(0); }

/* Touch devices have no hover: always show the button */
@media (hover: none) { .add-btn { opacity: 1; transform: none; } }

/* Respect user motion preference */ @media
(prefers-reduced-motion: reduce) {
* { transition: none !important; }
}
```

(d) Performance and user experience considerations

- **Performance:** animate only transform and opacity; these are GPU-accelerated and avoid layout reflow. Avoid animating width, height, top or margin.
- **will-change:** use sparingly (e.g. will-change: transform) to avoid excessive memory use.
- **Duration:** keep between 200 and 400 ms; longer feels sluggish.
- **Accessibility:** use :focus-within so keyboard users get the same effect; honour prefers-reduced-motion.
- **Mobile:** hover does not exist on touch screens, so the @media (hover: none) fallback keeps the button visible.
- **Consistency:** use one effect style across all cards to avoid distraction.

Question 3: Layout Selection – Flexbox vs Grid (4 Marks)

(a) Layout requirements of a dashboard

- **Rows:** top header bar, a main content region and optional footer.
- **Columns:** fixed-width sidebar plus a flexible main area; main area holds multi-column widgets.
- **Widgets / cards:** stat cards, charts and tables of different sizes (some spanning multiple columns/rows).
- **Alignment:** consistent gaps, equal-height cards, items aligned on both axes.
- **Responsiveness:** sidebar collapses and widgets stack on small screens.

(b) Flexbox vs Grid comparison

Criteria	Flexbox	CSS Grid
Dimension	One-dimensional (row OR column)	Two-dimensional (rows AND columns together)
Approach	Content-first: items decide their size	Layout-first: grid structure defined first
Spanning cells	Not natively supported	grid-column / grid-row span, named areas
Alignment	Excellent along one axis	Excellent on both axes
Gaps	gap supported	gap, row-gap and column-gap supported
Responsive	flex-wrap, flex-basis	repeat(auto-fit, minmax()) with no media queries
Best for	Navbars, button groups, centring, card rows	Page layouts, dashboards, galleries

(c) Chosen technique and justification

CSS Grid is chosen for the overall dashboard layout, with Flexbox used inside individual widgets. Justification: a dashboard is inherently two-dimensional (sidebar + header + content + widget grid). Grid allows named areas for readable structure, lets widgets span multiple rows/columns, keeps equal gutters, and produces a responsive widget grid with one line of code. Flexbox is then ideal for aligning items inside a card (icon + text, header + menu).

Dashboard using Grid

```
.dashboard {
display: grid; min-height: 100vh; gap: 16px;
  grid-template-columns: 220px 1fr;
grid-template-rows: 64px 1fr 48px;
  grid-template-areas:
"sidebar header" "sidebar
    main" "sidebar
    footer";
}
.header{ grid-area: header; }
.sidebar { grid-area: sidebar; background: #1f3864; color: #fff; }
.main  { grid-area: main; }
.footer{ grid-area: footer; }

/* Widget grid inside main: auto-responsive */
.widgets {
display: grid; gap: 16px;
grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
}
.widget.wide { grid-column: span 2; }
.widget { background: #fff; padding: 16px; border-radius: 10px;
          box-shadow: 0 1px 4px rgba(0,0,0,.15);
```

```
display: flex; flex-direction: column; justify-content: space-between; }

@media (max-width: 768px) {
  .dashboard { grid-template-columns: 1fr; grid-template-rows: auto;
    grid-template-areas: "header" "main" "sidebar" "footer"; }
.widget.wide { grid-column: span 1; }
}
```

(d) Scalability and responsiveness evaluation

- **Scalability:** adding widgets requires no layout changes; the auto-fit grid flows new cards automatically. Named areas make the layout easy to maintain.
- **Responsiveness:** minmax() lets columns grow and shrink; a single media query re-arranges the named areas on mobile.
- **Browser support:** Grid and Flexbox are supported by all modern browsers.
- **Limitations:** very deep nested grids can become complex; subgrid can be used to align nested content.

Question 4: Mobile-First Blog Layout (4 Marks)

(a) Key sections of a blog layout

- Header with logo and navigation
- Hero / featured post banner
- Main article list or article content (title, meta, image, text)
- Sidebar (search, categories, recent posts, tags)
- Comments / author bio / share buttons
- Footer (links, copyright, newsletter signup)

(b) Mobile-first layout using CSS

Mobile-first means the base CSS targets small screens (single column, fluid, readable), and min-width media queries progressively enhance the layout for bigger screens.

HTML

```
<header class="site-header"><h1>My Blog</h1><nav>Home | About | Contact</nav></header>
<div class="container">
<main class="posts">
<article class="post"><img src="a.jpg" alt="Post image">
<h2>Post title</h2><p class="meta">By Author · 23 Oct 2026</p>
<p>Post summary text...</p></article>
</main>
  <aside
class="sidebar"><h3>Categories</h3><ul><li>Tech</li><li>Travel</li></ul></aside>
</div>
<footer class="site-footer">&copy; 2026 My Blog</footer>
```

CSS (mobile base styles)

```
:root { --max: 1100px; }
* { box-sizing: border-box; }
body { font-family: Georgia, serif; font-size: 1rem; line-height: 1.7; color:
       #222; margin: 0; }
img { max-width: 100%; height: auto; }

.site-header { background: #1f3864; color: #fff; padding: 16px; text-align: center; }
.container { display: block; padding: 16px; max-width: var(--max); margin: auto; }
.post { margin-bottom: 24px; padding-bottom: 16px; border-bottom: 1px solid #ddd; }
.post h2 { font-size: 1.4rem; margin: 8px 0; }
.meta { color: #666; font-size: .85rem; }
.sidebar { background: #f5f5f5; padding: 16px; border-radius: 8px; }
.site-footer { background: #222; color: #fff; text-align: center; padding: 16px; }
```

(c) Scaling to larger screens with media queries

CSS (progressive enhancement)

```
/* Tablet: 600px and above - posts in 2 columns */
@media (min-width: 600px) {
.posts { display: grid; grid-template-columns: repeat(2, 1fr); gap: 20px; }
.post { border-bottom: none; }
}

/* Desktop: 992px and above - main + sidebar */ @media
(min-width: 992px) {
.container { display: grid; grid-template-columns: 3fr 1fr; gap: 32px; }
.posts { grid-template-columns: repeat(2, 1fr); }
.sidebar { align-self: start; position: sticky; top: 16px; }
.site-header { text-align: left; display: flex; justify-content: space-between; }
}

/* Large desktop: 1200px and above */
@media (min-width: 1200px) {
body { font-size: 1.1rem; }
.posts { grid-template-columns: repeat(3, 1fr); }
}
```

Screen	Layout
Mobile (< 600px)	Single column; sidebar below posts; full-width images.
Tablet (≥ 600px)	Posts in two columns.
Desktop (≥ 992px)	Main content with a sticky right sidebar (3:1 ratio).
Large (≥ 1200px)	Three post columns, larger text.

(d) Accessibility and readability evaluation

- **Semantic HTML:** header, main, article, aside and footer let screen readers navigate landmarks.
- **Readability:** line-height 1.7, relative units (rem) and optimum line length (about 60–75 characters) improve reading.
- **Contrast:** dark text on light background exceeds WCAG AA 4.5:1.
- **Images:** alt text on every image; max-width: 100% prevents overflow.
- **Zoom:** rem/fluid units allow text to scale to 200% without breaking layout.
- **Improvements:** add skip links, visible focus outlines, a dark-mode option (prefers-color-scheme), and lazy loading for images (loading="lazy").

Question 5: JavaScript Event Handling for Dynamic Menu (4 Marks)

(a) Appropriate JavaScript events

Event	Use in the menu
click	Toggle the hamburger menu; open/close dropdown on touch devices.
mouseenter / mouseleave	Show and hide dropdown on desktop hover.
keydown	Escape closes the menu; Enter/Space activates; arrow keys move between items.
focusin / focusout	Keep dropdown open while keyboard focus is inside.
resize	Reset the menu state when the window grows past the mobile breakpoint.
DOMContentLoaded	Run setup code after the DOM has loaded.

(b) Event handling logic for menu toggle

Logic (algorithm):

- Select the toggle button and the menu element.
- On button click, toggle the CSS class 'open' on the menu.
- Update aria-expanded to reflect the new state.
- On Escape key or click outside the menu, remove the 'open' class.
- On window resize above 768px, remove the class to reset the state.

(c) Implementation (click and hover)

HTML

```
<button id="menuToggle" aria-expanded="false" aria-controls="menu">&#9776; Menu</button>
<ul id="menu" class="menu">
<li><a href="#">Home</a></li>
<li class="has-sub">
<a href="#">Products</a>
    <ul class="submenu"><li><a href="#">Laptops</a></li><li><a
href="#">Phones</a></li></ul>
</li>
<li><a href="#">Contact</a></li>
</ul>
```

CSS

```
.menu { list-style: none; display: none; padding: 0; }
.menu.open { display: block; }
.submenu { display: none; list-style: none; padding-left: 16px; }
.has-sub.show > .submenu { display: block; }
@media (min-width: 769px) { .menu { display: flex; gap: 20px; } }
```

JavaScript

```
document.addEventListener('DOMContentLoaded', () => {
  const toggle = document.getElementById('menuToggle');
  const menu   = document.getElementById('menu');
const subs     = document.querySelectorAll('.has-sub');

  function closeMenu() {
    menu.classList.remove('open');
    toggle.setAttribute('aria-expanded', 'false');
  }

// CLICK: toggle main menu
  toggle.addEventListener('click', (e) => {
e.stopPropagation();
```

```
const isOpen = menu.classList.toggle('open');
    toggle.setAttribute('aria-expanded', isOpen);
  });

// CLICK on touch / HOVER on desktop for submenus
  subs.forEach(item => {
item.querySelector('a').addEventListener('click', (e) => {
      if (window.innerWidth <= 768) {       // mobile: tap to expand
        e.preventDefault();
item.classList.toggle('show');
      }
    });
item.addEventListener('mouseenter', () => {
if (window.innerWidth > 768) item.classList.add('show');
    });
item.addEventListener('mouseleave', () => {
if (window.innerWidth > 768) item.classList.remove('show');
    });
  });

// KEYBOARD: Escape closes the menu
  document.addEventListener('keydown', (e) => {
if (e.key === 'Escape') { closeMenu(); toggle.focus(); }
  });

// Click outside closes the menu (event delegation)
  document.addEventListener('click', (e) => {
if (!menu.contains(e.target) && e.target !== toggle) closeMenu();
  });

// RESIZE (debounced): reset state on desktop let
  timer;
  window.addEventListener('resize', () => {
    clearTimeout(timer);
timer = setTimeout(() => { if (window.innerWidth > 768) closeMenu(); }, 150);
  });
});
```

(d) Performance evaluation and improvements

- **Efficient DOM use:** toggling a CSS class triggers a single style change instead of editing many inline styles.
- **Debouncing:** the resize handler is debounced so it does not fire hundreds of times.
- **Event delegation:** one listener on the menu (instead of one on every link) reduces memory use for large menus.
- **Passive listeners:** use { passive: true } for scroll/touch listeners.
- **Avoid hover-only logic:** hover is replaced by click on touch devices.
- **Further improvements:** use CSS-only :hover/:focus-within for simple dropdowns, add arrow-key navigation, focus trapping for full-screen mobile menus, and remove listeners when components are destroyed.